

MILESTONE 3 ASSIGNMENT 3

Selenium Page Object Model in the Python Behave Framework

Name:	Roshni Ghosh Tagore
Enrollment No:	12023002028175
Course:	Python for Automation
Academic Year:	2023–2027
Department:	Computer Science and Engineering (Artificial Intelligence & Machine Learning)
Campus:	IEM Newtown

Department of Computer Science and Engineering (AI & ML)
Institute of Engineering & Management (IEM), Newtown Campus

1. PROBLEM STATEMENT

Directly embedding web element locators and low-level browser interactions inside BDD step definition files leads to high maintenance overhead and code duplication. Whenever a web UI element changes, multiple test step files require updates. This assignment addresses these design challenges by integrating the **Page Object Model (POM)** pattern into the **Python Behave BDD Framework**, decoupling UI locators and actions into modular class structures for optimized web application testing.

2. OBJECTIVE

The main objectives of this assignment are:

- Debug and optimize web application automations using the Behave framework and Gherkin scenarios.
- Implement a clean Page Object Model (POM) design pattern with reusable Base Page and Page Object classes.
- Encapsulate Web Elements and UI user interactions within dedicated page object modules.
- Leverage the Behave framework for data-driven testing using POM abstractions.
- Ensure high maintainability and scalability across browser automation test suites.

3. TOOLS, SOFTWARE, AND CONCEPTS USED

Tool / Concept	Description & Purpose
Python 3.x	Object-oriented programming language used to construct page classes, inheritance models, and test assertions.
Behave Framework	BDD automation engine that connects Gherkin feature scenarios to Python POM step implementations.
Selenium WebDriver	Browser automation API providing web element interaction methods, locator strategies, and explicit waits.
Page Object Model (POM)	Design pattern that creates an object repository for storing web element locators and page interaction behaviors.
Explicit Waits (WebDriverWait)	Synchronizes test execution with dynamic web page states to prevent flaky element lookup failures.

POM Architectural Advantage: Separating element locators from test logic ensures that changes to the application UI only require modifications in a single page class file.

4. SOURCE CODE & IMPLEMENTATION STEPS

Step 1: Base Page Wrapper (pages/base_page.py)

pages/base_page.py

Python Base Class

```
from selenium.webdriver.support.ui import WebDriverWait from selenium.webdriver.support
```

Step 2: Login Page Object Class (pages/login_page.py)

pages/login_page.py

Python Page Object

```
from selenium.webdriver.common.by import By from pages.base_page import BasePage class L
```

Step 3: Gherkin Feature File (features/pom_login.feature)

features/pom_login.feature

Gherkin Spec

```
Feature: Page Object Model Web Authentication Scenario Outline: Authenticate user using  
  " and password "  
  
  " Then the application should display message containing "  
  
  " Examples: | username | password | expected_message | | tomsmith | SuperSecretPasswo
```

Step 4: POM Step Definitions (features/steps/pom_login_steps.py)

features/steps/pom_login_steps.py

Python Logic

```
from behave import given, when, then from pages.login_page import LoginPage @given('the
```

5. OUTPUT EXPECTED

Feature: Page Object Model Web Authentication Scenario Outline: Authenticate user using I

Figure 3.1: Simple Lines of Execution Output for POM BDD Scenarios

6. RESULT

Execution Summary: The test suite successfully initialized the browser, loaded the target URL via the LoginPage object, executed data-driven login scenarios for both valid and invalid credentials, and verified flash message assertions accurately.

Observation: Integrating the Page Object Model (POM) pattern into the Python Behave framework isolates UI locator definitions from step execution logic. This ensures high code reusability, seamless maintainability, and clean separation of concerns.

Conclusion: The Page Object Model implementation within the Python Behave automation framework was completed successfully, passing all data-driven web automation scenarios.